Solution: Triplets with Smaller Sum

Problem Statement

Given an array **arr** of unsorted numbers and a target sum, count all triplets in it such that **arr[i] + arr[j] + arr[k] < target** where **i**, **j**, and **k** are three different indices. Write a function to return the count of such triplets.

Example 1:

```
Input: [-1, 0, 2, 3], target=3
Output: 2
Explanation: There are two triplets whose sum is less than the target:
[-1, 0, 3], [-1, 0, 2]
```

Example 2:

```
Input: [-1, 4, 2, 1, 3], target=5
Output: 4
Explanation: There are four triplets whose sum is less than the target:
[-1, 1, 4], [-1, 1, 3], [-1, 1, 2], [-1, 2, 3]
```

Solution

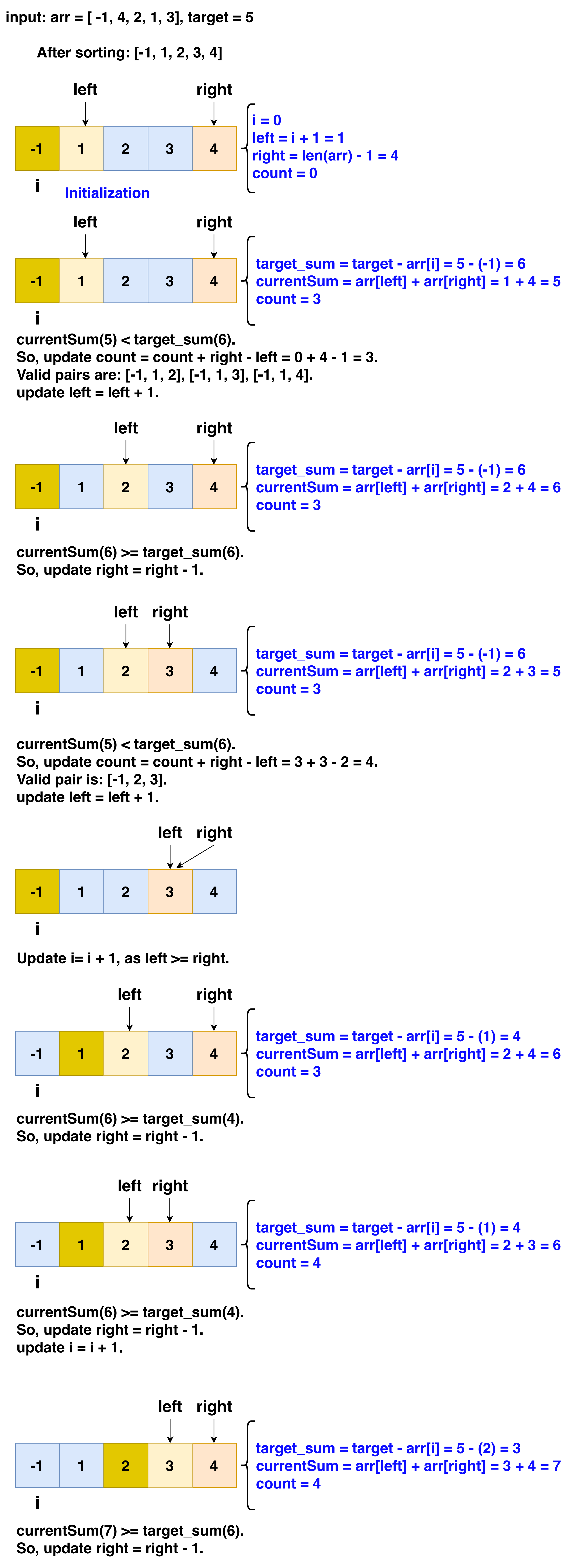
This problem follows the **Two Pointers** pattern and shares similarities with Triplet Sum to Zero. The only difference is that, in this problem, we need to find the triplets whose sum is less than the given target. To meet the condition `i != j != k` we need to make sure that each number is not used more than once.

Following a similar approach, first, we can sort the array and then iterate through it, taking one number at a time. Let's say during our iteration we are at number 'X', so we need to find 'Y' and 'Z' such that $X + Y + Z < target$. At this stage, our problem translates into finding a pair whose sum is less than "$target - X$" (as from the above equation $Y + Z < target - X$). We can use a similar approach as discussed in "Triplet Sum to Zero".

Here's a detailed walkthrough of the algorithm:

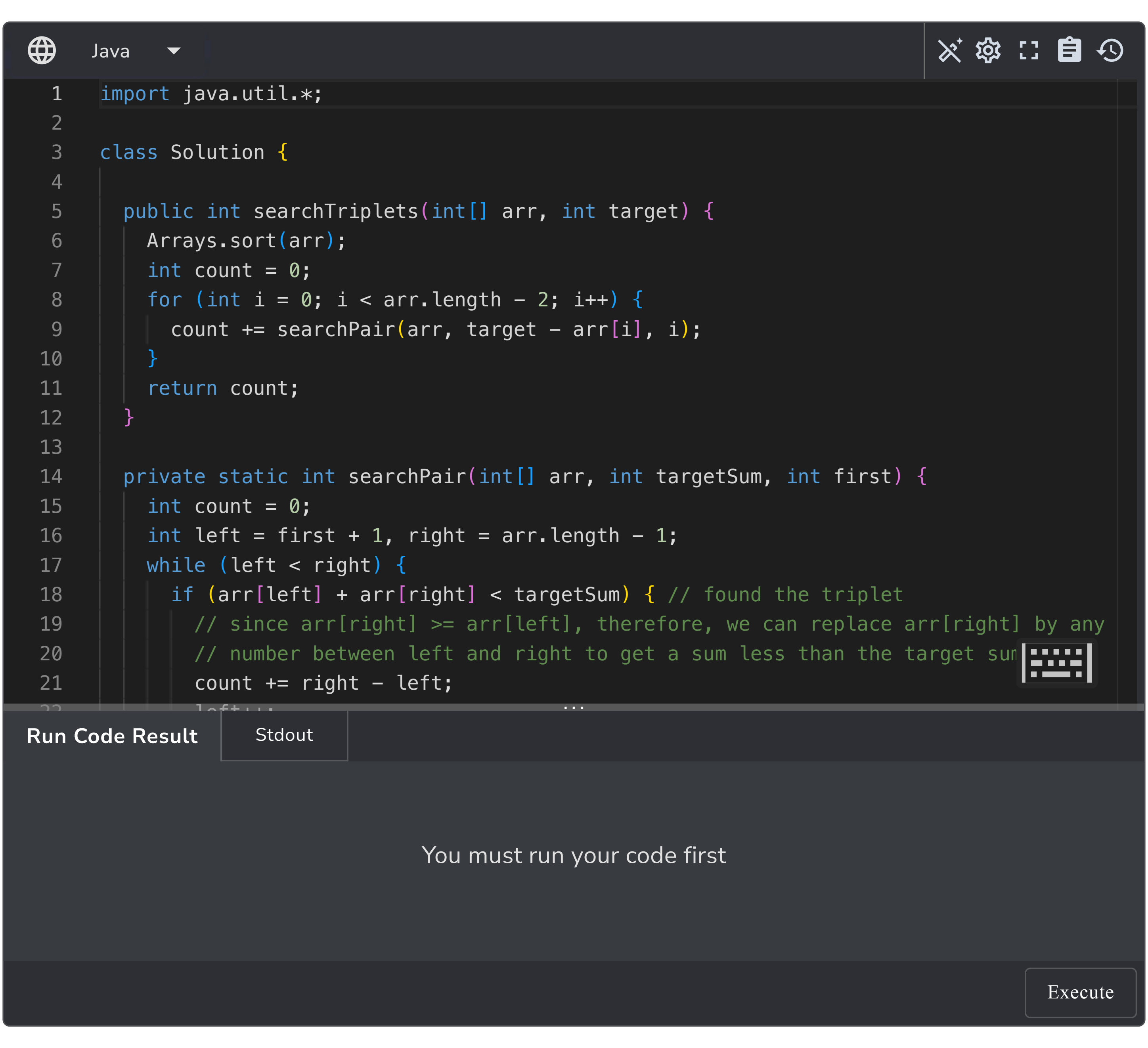
1. The method `searchTriplets` starts by sorting the input array `arr` in ascending order. Sorting is important as it allows us to move our pointers based on the sum we are getting and how close we are to the target sum.
2. The variable `count` is initialized to keep track of the total number of triplets found.
3. The function then iterates through `arr` using a `for` loop, stopping when it is two positions from the end of `arr` (`arr.length - 2`). This is because we are always looking for triplets and thus don't need to consider the last two positions in this loop.
4. Inside the `for` loop, we call the `searchPair` function with the array, the target value minus the current element, and the current index. This function will find all pairs within the array from index `first+1` to the end of the array whose sum with `arr[i]` is less than `target`. The return value, which is the count of such pairs, is added to `count`.
5. The `searchPair` function initializes two pointers: `left` to `first+1` and `right` to the last element in the array. It then enters a `while` loop that continues as long as `left` is less than `right`.
6. In the loop, if the sum of the elements at the `left` and `right` indices is less than `targetSum`, this means we have found a valid pair, because adding `arr[first]` would still result in a sum less than `target`. Since the array is sorted, all the elements between `left` and `right` with `arr[first]` will also form valid triplets. So, we add all these pairs to our `count` by adding `right - left` to `count`.
7. We then increment `left` to move towards higher numbers in the array.
8. If the sum of the elements at `left` and `right` is not less than `targetSum`, we need a smaller sum. Since the array is sorted, to achieve a smaller sum, we need to reduce the value of the larger number. Hence, we decrement `right`.
9. This process repeats until `left` and `right` cross, at which point we have examined all possible pairs for our current value of `first`.
10. Once `searchPair` has processed all possible pairs for the given `first` index, it returns the `count` of valid pairs.
11. The loop in `searchTriplets` continues until we have tried every possible starting point for our triplet.
12. Once all possible triplets have been considered, the `searchTriplets` function returns `count`, the total number of triplets whose sum is less than `target`.

Let's visualize the example 2 via diagram below.


input: arr = [-1, 4, 2, 1, 3], target = 5
After sorting: [-1, 1, 2, 3, 4]
left
right
-1
1
2
3
4
i
i = 0
left = i + 1 = 1
right = len(arr) - 1 = 4
count = 0
Initialization
target_sum = target - arr[i] = 5 - (-1) = 6
currentSum = arr[left] + arr[right] = 1 + 4 = 5
count = 3
currentSum(5) < target_sum(6).
So, update count = count + right - left = 0 + 4 - 1 = 3.
Valid pairs are: [-1, 1, 2], [-1, 1, 3], [-1, 1, 4].
update left = left + 1.
target_sum = target - arr[i] = 5 - (-1) = 6
currentSum = arr[left] + arr[right] = 2 + 4 = 6
count = 3
currentSum(6) >= target_sum(6).
So, update right = right - 1.
target_sum = target - arr[i] = 5 - (-1) = 6
currentSum = arr[left] + arr[right] = 2 + 3 = 5
count = 3
currentSum(5) < target_sum(6).
So, update count = count + right - left = 3 + 3 - 2 = 4.
Valid pair is: [-1, 2, 3].
update left = left + 1.
Update i= i + 1, as left >= right.
target_sum = target - arr[i] = 5 - (1) = 4
currentSum = arr[left] + arr[right] = 2 + 4 = 6
count = 3
currentSum(6) >= target_sum(4).
So, update right = right - 1.
target_sum = target - arr[i] = 5 - (1) = 4
currentSum = arr[left] + arr[right] = 2 + 3 = 6
count = 4
currentSum(6) >= target_sum(4).
So, update right = right - 1.
update i = i + 1.
target_sum = target - arr[i] = 5 - (2) = 3
currentSum = arr[left] + arr[right] = 3 + 4 = 7
count = 4
currentSum(7) >= target_sum(6).
So, update right = right - 1.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Solution {

  public int searchTriplets(int[] arr, int target) {
    Arrays.sort(arr);
    int count = 0;
    for (int i = 0; i < arr.length - 2; i++) {
      count += searchPair(arr, target - arr[i], i);
    }
    return count;
  }

  private static int searchPair(int[] arr, int targetSum, int first) {
    int count = 0;
    int left = first + 1, right = arr.length - 1;
    while (left < right) {
      if (arr[left] + arr[right] < targetSum) { // found the triplet
        // since arr[right] >= arr[left], therefore, we can replace arr[right] by any
        // number between left and right to get a sum less than the target sum
        count += right - left;
```

You must run your code first

Time Complexity

Sorting the array will take $O(N * logN)$. The searchPair() will take $O(N)$. So, overall searchTriplets() will take $O(N * logN + N^2)$, which is asymptotically equivalent to $O(N^2)$.

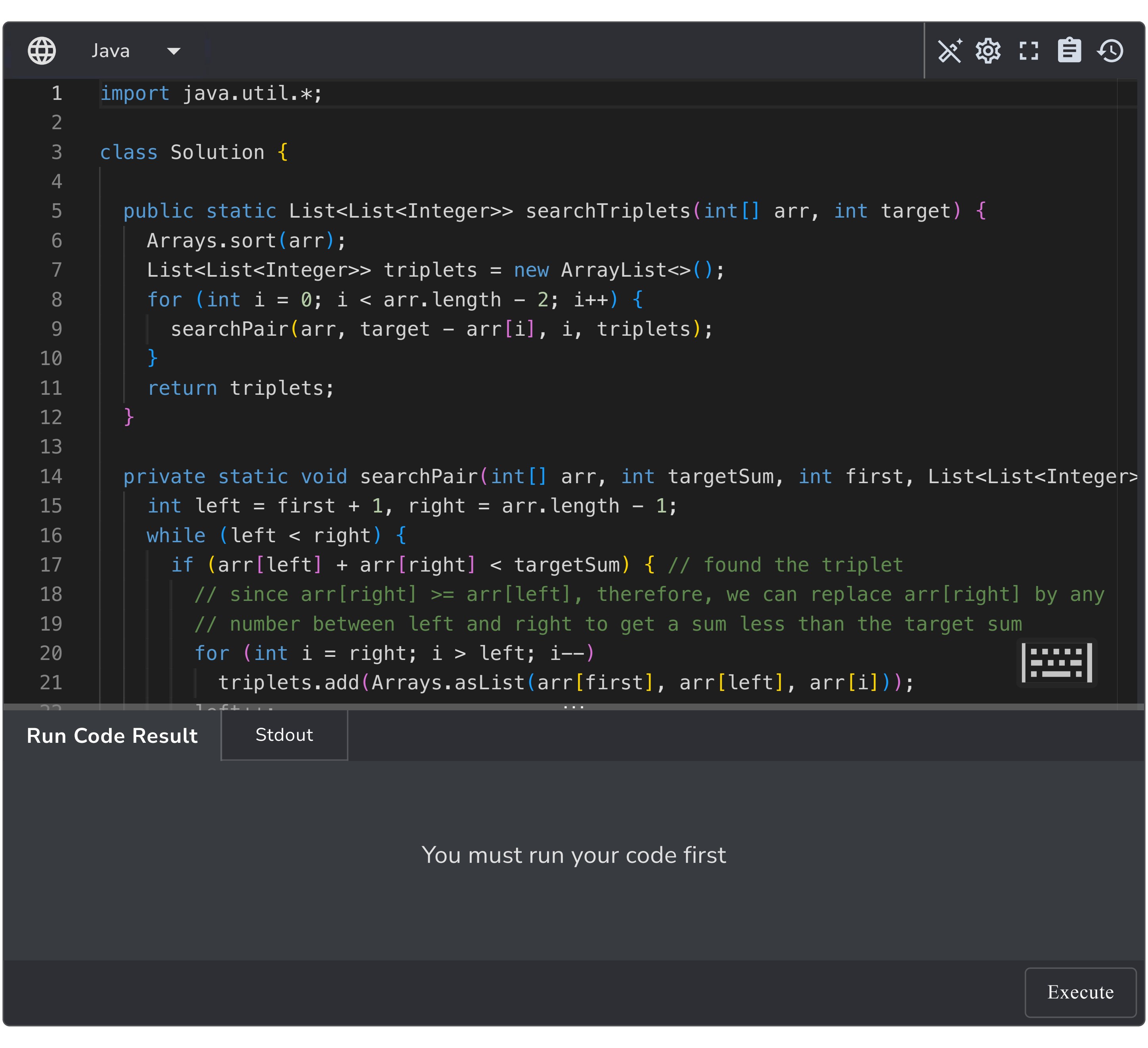
Space Complexity

The space complexity of the above algorithm will be $O(N)$ which is required for sorting if we are not using an in-place sorting algorithm.

Similar Problems

Problem:
Write a function to return the list of all such triplets instead of the count. How will the time complexity change in this case?

Solution:
Following a similar approach, we can create a list containing all the triplets. Here is the code::



```java
import java.util.*;

class Solution {

  public static List<List<Integer>> searchTriplets(int[] arr, int target) {
    Arrays.sort(arr);
    List<List<Integer>> triplets = new ArrayList<>();
    for (int i = 0; i < arr.length - 2; i++) {
      searchPair(arr, target - arr[i], i, triplets);
    }
    return triplets;
  }

  private static void searchPair(int[] arr, int targetSum, int first, List<List<Integer>
    int left = first + 1, right = arr.length - 1;
    while (left < right) {
      if (arr[left] + arr[right] < targetSum) { // found the triplet
        // since arr[right] >= arr[left], therefore, we can replace arr[right] by any
        // number between left and right to get a sum less than the target sum
        for (int i = right; i > left; i--)
          triplets.add(Arrays.asList(arr[first], arr[left], arr[i]));
```

You must run your code first

Another simpler approach could be to check every triplet of the array with three nested loops and create a list of triplets that meet the required condition.

Time Complexity

Sorting the array will take $O(N * logN)$. The `searchPair()`, in this case, will take $O(N^2)$ the main while loop will run in $O(N)$ but the nested for loop can also take $O(N)$ - this will happen when the target sum is bigger than every triplet in the array. So, overall searchTriplets() will take $O(N * logN + N^3)$, which is asymptotically equivalent to $O(N^3)$

Space Complexity

Ignoring the space required for the output array, the space complexity of the above algorithm will be $O(N)$ which is required for sorting.

← Previous
Triplets With Smaller Sum (Medium)

Next →
Subarrays With Product Less Than A Target (Medium)

☐ Mark as Completed